

Notas de migración: Pydantic v1 a v2

Cambio de `.dict()/json()` a `.model_dump()/model_dump_json()`

En Pydantic v2, los métodos `.dict()` y `.json()` de una instancia de `BaseModel` quedaron deprecados en favor de `.model_dump()` y `.model_dump_json()`. Funcionalmente son casi equivalentes, pero `.model_dump()` acepta el parámetro `mode='json'` para forzar que los tipos no serializables nativamente en JSON (como `datetime` o `Decimal`) se conviertan a su representación de string, en vez de quedar como objetos Python.

Los validadores cambian de decorador

El decorador `@validator` de v1 se reemplaza por `@field_validator` en v2, y su firma cambia: ya no recibe todos los 'values' anteriores por default (hay que usar `@model_validator` si se necesita validar en base a otros campos). `@root_validator` se reemplaza por `@model_validator(mode='before')` o `mode='after')` según en qué momento del parseo se necesite ejecutar la validación.

Config de clase interna a `model_config`

En v1 la configuración de un modelo se declaraba con una clase interna `'class Config'`. En v2 se reemplaza por un atributo de clase `'model_config = ConfigDict(...)'`. Por ejemplo, `'orm_mode = True'` de v1 pasa a llamarse `'model_config = ConfigDict(from_attributes=True)'` en v2.

Rendimiento: el núcleo ahora está escrito en Rust (pydantic-core)

Pydantic v2 delega el parseo y la validación a `pydantic-core`, una librería escrita en Rust. En la práctica esto significa mejoras de rendimiento de varias veces respecto a v1 para la mayoría de los modelos, sin cambios en el código de la aplicación más allá de la migración de API ya mencionada.

Modelos inmutables y 'frozen'

Para declarar un modelo inmutable (que lance una excepción si se intenta reasignar un campo después de creado), en v2 se usa `'model_config = ConfigDict(frozen=True)'`, en lugar del `'allow_mutation = False'` de v1.

Compatibilidad hacia atrás

Pydantic v2 incluye un módulo de compatibilidad 'pydantic.v1' que permite importar el comportamiento de v1 dentro de un proyecto ya migrado a v2, útil para migrar gradualmente módulo por módulo en lugar de todo el proyecto de una sola vez.